
EMBEDDED HACKING

Reverse Engineering the Raspberry Pi Pico 2

OPERATION IRON CHOIR - Instructor Solution Key

ARM Cortex-M33 • GDB • Ghidra

September 27, 2026

For educational and defensive security research only

The task and criterion headings in this key are word-for-word identical to `ACT-VII-R.md`, so a student can match each criterion one to one.

Artifact Identity

The instructor-issued artifact hashes are:

ACT-VII.bin	c9996a62a7157e7aed43b3caf4e33348a87b8adfa4bbf67e1f1ff94df89cef42
ACT-VII.uf2	79b92868b5218572dfe41555e5b5157a01f1f277724ac5d7524de657f2def8e7
ACT-VII_fixed.bin	b5fe79ce517f820449ad1dc2ecf628e6702891174abc9c33e923556d7f8c7534
ACT-VII_fixed.uf2	6aa0f6742ae2baf8ab44f68a6d5bb7d26fa19901f3add2bc2a0311ddabd442f4

Machine check: `python scripts/verify_ctf.py` returns 10/10 checks passed against the shipped and corrected images. It asserts the four byte pairs, that only those four offsets differ, and the `ACT-VII.bin` and `ACT-VII_fixed.bin` SHA-256 values. Both `.bin` images are 51,196 bytes and both `.uf2` images are 102,912 bytes.

The four sabotage sites (summary):

Defect	Function	File offset	VA	Compromised	Correct
1 C2 check-in	<code>implant_tick</code> (inlined <code>implant_check_in</code>)	<code>0xA47F</code>	<code>0x1000A47F</code>	<code>0xB9</code>	<code>0xB1</code>
2 Task handler	<code>implant_handle_command</code> (inlined <code>implant_tasking_frame</code>)	<code>0xA2A9</code>	<code>0x1000A2A9</code>	<code>0xB9</code>	<code>0xB1</code>
3 Bot marker	<code>implant_init</code> (inlined <code>implant_infect</code>)	<code>0xA3EF</code>	<code>0x1000A3EF</code>	<code>0xB9</code>	<code>0xB1</code>
4 Fault-clear authorization	<code>control_handle_frame</code>	<code>0x7541</code>	<code>0x10007541</code>	<code>0xB9</code>	<code>0xB1</code>

Task 1: Setup and Initial Analysis (10 points)

Solution

Ghidra Setup. Import `ACT-VII.bin` as Raw Binary, language ARM Cortex 32 little endian default, base address `0x10000000`, then run auto-analysis. The Ghidra project name is `IronChoir_Investigation`. Because every defect is a same-size in-place byte patch, the file offset and the VA differ by exactly `0x10000000` (`VA = offset + 0x10000000`).

Vector Table Decoding. First 32 bytes of `ACT-VII.bin`:

```
00 20 08 20 5D 01 00 10 1B 01 00 10 1D 01 00 10
11 01 00 10 11 01 00 10 11 01 00 10 11 01 00 10
```

Evidence	Answer
Vector table base	<code>0x10000000</code>
Initial SP	<code>0x20082000</code>

Evidence	Answer
Reset handler (as stored)	0x1000015D
Reset instruction address	0x1000015C

The stored reset handler address has bit 0 set, selecting Thumb mode. Clearing bit 0 gives the real entry 0x1000015C.

Entry and Monitor Loop. From ACT-VII-main-disasm.txt :

```

10000234 <main>:
10000234:    b508          push    {r3, lr}
10000236:    f003 fa93     bl      10003760 <stdio_init_all>
1000023a:    4807          ldr     r0, [pc, #28] @ (10000258 <main+0x24>)
1000023c:    f003 fada     bl      100037f4 <__wrap_puts>
10000240:    f006 f900     bl      10006444 <monitor_init>
10000244:    b110          cbz     r0, 1000024c <main+0x18>
10000246:    f006 f9dd     bl      10006604 <monitor_step>
1000024a:    e7fc          b.n     10000246 <main+0x12>

```

Element	Address
main	0x10000234
monitor_init	0x10006444
monitor_step	0x10006604

Module Map. Anchors for the stripped image:

Module	Anchor function	Address
Entry	main	0x10000234
Monitor / andon state machine	monitor_init	0x10006444
Monitor / andon state machine	monitor_step	0x10006604
Control (sealed fault path)	control_handle_frame	0x100074D4
Control (applied command)	control_command	0x10007568
Control (applied zone)	control_zone	0x10007574
Diverter (actuator)	diverter_init	0x10007580
Diverter (actuator)	diverter_apply_command	0x10007598
Diverter (actuator)	diverter_tick	0x100075CC
Diverter (actuator)	diverter_fail_safe	0x1000760C
Diverter (actuator)	diverter_retract	0x1000A8F4
Diverter (actuator)	diverter_deploy	0x1000A910
Andon authorization	andon_auth_init	0x10007628

Module	Anchor function	Address
Andon authorization	andon_auth_set_key	0x1000763C
Andon authorization	andon_auth_apply	0x10007684
Implant	implant_tick	0x1000A468
Implant	implant_handle_command	0x1000A2A4
Implant	implant_init	0x1000A3AC
Implant	implant_infected	0x1000A290
Radio	radio_send_frame	0x1000A578
Crypto	envelope_open_hex	0x10007928
Tower light	status_led_show	0x1000A874

Grading Rubric (1-to-1 Mapping)

Criterion	Points	Full Credit (Answer Key)
[DOCUMENT] Ghidra project created with the correct name and settings	2	Project IronChoir_Investigation , raw binary import
[DOCUMENT] Processor configured as ARM Cortex 32 little endian default	2	Screenshot shows the correct processor
[DOCUMENT] Base address set to 0x10000000	2	Base 0x10000000
[DOCUMENT] Vector table, initial stack pointer, and reset handler identified	2	Base 0x10000000 , initial SP 0x20082000 , reset handler 0x1000015D
[DOCUMENT] main and the andon monitor state machine (monitor_step) addresses identified	1	main 0x10000234 , monitor_step 0x10006604
[DOCUMENT] Module map identifies the diverter, control, andon_auth, implant, and monitor anchors	1	At least one correct anchor per module

Instructor Notes & Assembly

- Confirm the Ghidra import used Raw Binary , ARM Cortex 32 little endian default , base 0x10000000 , and that auto-analysis completed before any address was read. In the language dialog the student must search Cortex and pick the ARM Cortex 32 little endian default entry.
- Accept either the Import Results Summary or the Program Information window as proof of the name, language, and base address.
- The stored reset handler 0x1000015D is odd because bit 0 selects Thumb; clearing it gives 0x1000015C .
- Always say reset handler , never reset pointer .
- The vector table is identical in the compromised and corrected images because no defect touches it.
- The module map is graded on coverage, not on exhaustive function recovery: one correctly named anchor per module is sufficient. implant_infect , implant_check_in , and implant_tasking_frame are inlined and have no standalone symbol.

Task 2: Bug #1 The C2 Check-In (20 points)

Solution

Locate the branch. `implant_tick` starts at `0x1000A468` and the inlined `implant_check_in` gate is at file offset `0xA47F` (VA `0x1000A47F`). The corrected image is:

```

1000a468 <implant_tick>:
1000a468: 4a15      ldr     r2, [pc, #84]    @ (1000a4c0 <implant_tick+0x58>)
1000a46a: 4916      ldr     r1, [pc, #88]    @ (1000a4c4 <implant_tick+0x5c>)
1000a46c: 6813      ldr     r3, [r2, #0]
1000a46e: 7809      ldrb    r1, [r1, #0]
1000a470: 3301      adds    r3, #1
1000a472: 6013      str     r3, [r2, #0]
1000a474: b311      cbz     r1, 1000a4bc <implant_tick+0x54>
1000a476: 079a      lsls    r2, r3, #30
1000a478: d120      bne.n   1000a4bc <implant_tick+0x54>
1000a47a: 4b13      ldr     r3, [pc, #76]    @ (1000a4c8 <implant_tick+0x60>)
1000a47c: 781b      ldrb    r3, [r3, #0]
1000a47e: b1eb      cbz     r3, 1000a4bc <implant_tick+0x54>
1000a480: 4b12      ldr     r3, [pc, #72]    @ (1000a4cc <implant_tick+0x64>)
1000a482: 781b      ldrb    r3, [r3, #0]
1000a484: b1d3      cbz     r3, 1000a4bc <implant_tick+0x54>
1000a486: f04f 23e0  mov.w   r3, #3758153728 @ 0xe000e000
1000a48a: f8d3 3df0  ldr.w   r3, [r3, #3568] @ 0xdf0
1000a48e: 079b      lsls    r3, r3, #30
1000a490: d114      bne.n   1000a4bc <implant_tick+0x54>
1000a492: 23b7      movs    r3, #183        @ 0xb7
1000a494: b510      push    {r4, lr}
1000a496: 4c0e      ldr     r4, [pc, #56]    @ (1000a4d0 <implant_tick+0x68>)
1000a498: b082      sub     sp, #8
1000a49a: 4669      mov     r1, sp
1000a49c: 2206      movs    r2, #6
1000a49e: 480d      ldr     r0, [pc, #52]    @ (1000a4d4 <implant_tick+0x6c>)
1000a4a0: f8ad 3004  strh.w  r3, [sp, #4]
1000a4a4: 9400      str     r4, [sp, #0]
1000a4a6: f000 f867  bl      1000a578 <radio_send_frame>

```

Instruction decode. `ldr r2, [pc, #84]` loads the tick counter at `0x20013714` (literal at `0x1000A4C0`), and `ldr r1, [pc, #88]` loads the armed flag at `0x20013CF7` (literal at `0x1000A4C4`). The tick counter advances at `0x1000A470`, the low two bits of the tick select the four-tick interval at `0x1000A476`, and `ldr r3, [pc, #76]` loads the check-in gate at `0x20013CFA` (literal at `0x1000A4C8`). The branch at `0x1000A47E` decides whether the check-in may run. The correct code does nothing when the check-in gate is clear, so the branch at `0x1000A47E` must be `cbz` (`0xB1`) to the `0x1000A4BC` return. When the gate is set, the path tests the check-in enable flag at `0x20013CF9`, reads CoreDebug `DHCSR`, builds the six-byte frame from the `C2V1` magic at `0x1000A4D0` (`0x31563243`), the bot id `0xB7` at `0x1000A492`, and a zero task byte, then calls `radio_send_frame` at `0x1000A4A6`. The condition byte is the high byte at `0x1000A47F`.

Address	File offset	Compromised byte	Compromised instruction	Correct byte	Correct instruction
<code>0x1000A47F</code>	<code>0xA47F</code>	<code>0xB9</code>	<code>cbnz r3, 0x1000A4BC</code>	<code>0xB1</code>	<code>cbz r3, 0x1000A4BC</code>

Patch.

File Offset	VA	Original Bytes	Patched Bytes
<code>0xA47F</code>	<code>0x1000A47F</code>	EB B9	EB B1

Why the station no longer registers. The check-in frame is the 4-byte `C2V1` magic (`0x31563243` little-endian), the bot identifier `0xB7`, and a zero task byte, sent to the gateway address on the control radio. Under the compromised `cbnz`, the gate is inverted: the fall-through registration path is taken when the check-in gate is clear, so the bot announces itself every four ticks. After the patch, `cbz` returns while the gate is clear, so no frame is built and no check-in is sent. Because the encoder reads the raw control payload and emits on the same radio as the sealed fault path, no cryptographic control on the envelope can see or stop it; the only fix is the gate itself.

Grading Rubric (1-to-1 Mapping)

Criterion	Points	Full Credit (Answer Key)
[DOCUMENT] Located the C2 check-in branch at <code>0x1000A47F</code>	5	Address and function (<code>implant_tick</code> , inlined <code>implant_check_in</code>) identified
[DOCUMENT] Documented the C2V1 check-in frame and the bot id <code>0xB7</code>	5	4-byte <code>C2V1</code> magic, bot id <code>0xB7</code> , six-byte frame, four-tick interval
[DOCUMENT & PATCH] Patched <code>0xB9</code> to <code>0xB1</code> so the node does not register	7	Byte <code>0xB9</code> changed to <code>0xB1</code>
[DOCUMENT] Explained why the raw check-in needs no cipher and hides beneath the sealed path	3	Raw path under the sealed envelope and a listener that holds no key

Instructor Notes & Assembly

- The condition byte is the high byte at `0xA47F`; the correct halfword is `b1eb` for `cbz` and the compromised halfword is `b9eb`, so the on-disk bytes are `EB B1` for the fix and `EB B9` for the compromise.
- `cbz` branches when the register is zero (the gate is clear); `cbnz` branches when it is non-zero. The register holds the check-in gate, so the semantics are "do not register while the gate is clear".
- The check-in magic is `ANDON_IMPLANT_C2_MAGIC` (`C2V1`), `ANDON_IMPLANT_C2_MAGIC_LEN` is `4`, the bot id is `ANDON_IMPLANT_BOT_ID` (`0xB7`), the frame is six bytes (`ANDON_IMPLANT_TASK_FRAME_LEN`), and the interval is `ANDON_IMPLANT_TICK_INTERVAL` (`4`).
- The check-in gate is at `0x20013CFA`, the enable flag at `0x20013CF9`, the armed flag at `0x20013CF7`, the checked-in flag at `0x20013CF8`, and the check-in count at `0x2001370C`.
- Full credit requires both the byte change and a correct statement of the lesson: the check-in is not a cipher break, it is a separate channel beside the sealed protocol.

Task 3: Bug #2 The Task Handler (20 points)

Solution

Locate the branch. `implant_handle_command` starts at `0x1000A2A4` and the inlined `implant_tasking_frame` gate is at file offset `0xA2A9` (VA `0x1000A2A9`). The corrected image is:

```

1000a2a4 <implant_handle_command>:
1000a2a4: 4b36      ldr      r3, [pc, #216] @ (1000a380 <implant_handle_command+0xdc>)
1000a2a6: 781b      ldrb     r3, [r3, #0]
1000a2a8: b193      cbz      r3, 1000a2d0 <implant_handle_command+0x2c>
1000a2aa: 4b36      ldr      r3, [pc, #216] @ (1000a384 <implant_handle_command+0xe0>)
1000a2ac: 781b      ldrb     r3, [r3, #0]
1000a2ae: b17b      cbz      r3, 1000a2d0 <implant_handle_command+0x2c>
1000a2b0: f04f 23e0  mov.w   r3, #3758153728 @ 0xe000e000
1000a2b4: f8d3 3df0  ldr.w   r3, [r3, #3568] @ 0xdf0
1000a2b8: 079b      lsls     r3, r3, #30
1000a2ba: d109      bne.n   1000a2d0 <implant_handle_command+0x2c>
1000a2bc: b140      cbz      r0, 1000a2d0 <implant_handle_command+0x2c>
1000a2be: 2905      cmp     r1, #5
1000a2c0: d906      bls.n   1000a2d0 <implant_handle_command+0x2c>
1000a2c2: 7803      ldrb     r3, [r0, #0]
1000a2c4: 2b43      cmp     r3, #67 @ 0x43
1000a2c6: d103      bne.n   1000a2d0 <implant_handle_command+0x2c>
1000a2c8: 7843      ldrb     r3, [r0, #1]
1000a2ca: 1c42      adds    r2, r0, #1
1000a2cc: 2b32      cmp     r3, #50 @ 0x32
1000a2ce: d000      beq.n   1000a2d2 <implant_handle_command+0x2e>
1000a2d0: 4770      bx      lr
1000a2d2: f812 3f01  ldrb.w  r3, [r2, #1]!
1000a2d6: 2b56      cmp     r3, #86 @ 0x56
1000a2d8: d1fa      bne.n   1000a2d0 <implant_handle_command+0x2c>
1000a2da: 7853      ldrb     r3, [r2, #1]
1000a2dc: 2b31      cmp     r3, #49 @ 0x31
1000a2de: d1f7      bne.n   1000a2d0 <implant_handle_command+0x2c>
1000a2e0: 2101      movs    r1, #1
1000a2e2: b530      push    {r4, r5, lr}
1000a2e4: 4b28      ldr      r3, [pc, #160] @ (1000a388 <implant_handle_command+0xe4>)
1000a2e6: 4a29      ldr      r2, [pc, #164] @ (1000a38c <implant_handle_command+0xe8>)
1000a2e8: 781b      ldrb     r3, [r3, #0]
1000a2ea: b0c1      sub     sp, #260 @ 0x104
1000a2ec: 7944      ldrb     r4, [r0, #5]
1000a2ee: 7011      strb     r1, [r2, #0]

```

Instruction decode. `ldr r3, [pc, #216]` loads the task gate at `0x20013CFD` (literal at `0x1000A380`), and the branch at `0x1000A2A8` decides whether the handler may run. The correct code does nothing when the task gate is clear, so the branch at `0x1000A2A8` must be `cbz (0xB1)` to the `0x1000A2D0` return. When the gate is set, the path tests the tasking enable flag at `0x20013CFE` (literal at `0x1000A384`), reads CoreDebug_DHCSR, requires a frame longer than five bytes, and matches the raw magic byte by byte: `0x43 C`, `0x32 2`, `0x56 V`, `0x31 1`. On a match it arms the payload, writes the bot marker, records the task, and executes it. The condition byte is the high byte at `0x1000A2A9`.

Address	File offset	Compromised byte	Compromised instruction	Correct byte	Correct instruction
0x1000A2A9	0xA2A9	0xB9	cbnz r3, 0x1000A2D0	0xB1	cbz r3, 0x1000A2D0

Patch.

File Offset	VA	Original Bytes	Patched Bytes
0xA2A9	0x1000A2A9	93 B9	93 B1

Why the station no longer executes a task. Under the compromised `cbnz`, the gate is inverted: the fall-through tasking path is taken when the task gate is clear, so a C2V1 frame runs the benign tasks. The task byte is read at `0x1000A2EC` and dispatched: `ANDON_IMPLANT_TASK_BLINK (0x01)` calls `status_led_show`, `ANDON_IMPLANT_TASK_LOG (0x02)` prints a synthetic log line, and `ANDON_IMPLANT_TASK_REPORT (0x03)` sends a report frame back to the listener with `radio_send_frame`. After the patch, `cbz` returns while the gate is clear, so

the magic is never matched and no task is executed. Stopping the task handler is a separate control from stopping the check-in: the check-in announces the bot, and the handler obeys, so closing only one leaves either a silent bot that still answers or a loud bot that does nothing.

Grading Rubric (1-to-1 Mapping)

Criterion	Points	Full Credit (Answer Key)
[DOCUMENT] Located the task handler branch at 0x1000A2A9	5	Address and function (<code>implant_handle_command</code> , inlined <code>implant_tasking_frame</code>) identified
[DOCUMENT] Documented the TASK_BLINK, TASK_LOG, TASK_REPORT handler	5	Magic match before the sealed path, three benign tasks, unknown task ignored
[DOCUMENT & PATCH] Patched 0xB9 to 0xB1 so no remote task executes	7	Byte <code>0xB9</code> changed to <code>0xB1</code>
[DOCUMENT] Explained why stopping the task handler is separate from stopping the check-in	3	The handler obeys, the check-in announces

Instructor Notes & Assembly

- The condition byte is the high byte at `0xA2A9` ; the correct halfword is `b193` for `cbz` and the compromised halfword is `b993` , so the on-disk bytes are `93 B1` for the fix and `93 B9` for the compromise.
- The magic bytes in the compare are `0x43` (`C`), `0x32` (`2`), `0x56` (`V`), and `0x31` (`1`), which spell `C2V1` in order.
- The task gate is at `0x20013CFD` , the tasking enable flag at `0x20013CFE` , the last task at `0x20013CFB` , and the task count at `0x20013710` .
- The benign tasks are `ANDON_IMPLANT_TASK_BLINK` (`0x01`), `ANDON_IMPLANT_TASK_LOG` (`0x02`), and `ANDON_IMPLANT_TASK_REPORT` (`0x03`); an unrecognized task code is recorded and ignored.
- The `implant_tasking_frame` path is inlined into `implant_handle_command` ; there is no standalone symbol in the stripped image.
- Full credit requires both the byte change and a correct statement of the lesson: a device that runs data it did not authorize is obedient to whoever sent the data, even when the task is harmless.

Task 4: Bug #3 The Bot Marker (20 points)

Solution

Locate the branch. The `implant_infect` path is inlined into `implant_init` (starts at `0x1000A3AC`). The marker gate is at file offset `0xA3EF` (VA `0x1000A3EF`). The corrected image is:

```

1000a3ac <implant_init>:
1000a3e0:      4b1f          ldr     r3, [pc, #124] @ (1000a460 <implant_init+0xb4>)
1000a3e2:      f893 c000     ldrb.w  ip, [r3]
1000a3e6:      f1bc 0fc7     cmp.w   ip, #199 @ 0xc7
1000a3ea:      d01f          beq.n   1000a42c <implant_init+0x80>
1000a3ec:      780a          ldrb    r2, [r1, #0]
1000a3ee:      b1da          cbz     r2, 1000a428 <implant_init+0x7c>
1000a3f0:      781b          ldrb    r3, [r3, #0]
1000a3f2:      2bc7          cmp     r3, #199 @ 0xc7
1000a3f4:      d018          beq.n   1000a428 <implant_init+0x7c>
1000a3f6:      f3ef 8410     mrs     r4, PRIMASK
1000a3fa:      b672          cpsid   i
1000a3fc:      22ff          movs    r2, #255 @ 0xff
1000a3fe:      f10d 0001     add.w   r0, sp, #1
1000a402:      4611          mov     r1, r2
1000a404:      f000 fa94     bl      1000a930 <memset>
1000a408:      23c7          movs    r3, #199 @ 0xc7
1000a40a:      f44f 5180     mov.w   r1, #4096 @ 0x1000
1000a40e:      4815          ldr     r0, [pc, #84] @ (1000a464 <implant_init+0xb8>)
1000a410:      f88d 3000     strb.w  r3, [sp]
1000a414:      f000 fbdc     bl      1000abd0 <__flash_range_erase_veneer>
1000a418:      f44f 7280     mov.w   r2, #256 @ 0x100
1000a41c:      4669          mov     r1, sp
1000a41e:      4811          ldr     r0, [pc, #68] @ (1000a464 <implant_init+0xb8>)
1000a420:      f000 fbba     bl      1000ab98 <__flash_range_program_veneer>

```

Instruction decode. `ldr r3, [pc, #124]` loads the reserved sector at `0x103FF000` (literal at `0x1000A460`), and `ldrb.w ip, [r3]` reads the marker byte. The compare at `0x1000A3E6` detects an already-present `0xC7` marker and records the bot as infected at `0x1000A42C`. `ldrb r2, [r1, #0]` loads the marker gate at `0x20013CFC` (literal at `0x1000A448`), and the branch at `0x1000A3EE` decides whether the marker may be written. The correct code writes no marker when the gate is clear, so the branch at `0x1000A3EE` must be `cbz` (`0xB1`) to the `0x1000A428` return. When the gate is set, a second check at `0x1000A3F2` guards the write, and the Pico SDK flash sequence runs: `strb.w r3, [sp]` stages `0xC7` (`movs r3, #199` at `0x1000A408`), then `flash_range_erase` at `0x1000A414` and `flash_range_program` at `0x1000A420` program the sector. The condition byte is the high byte at `0x1000A3EF`.

Address	File offset	Compromised byte	Compromised instruction	Correct byte	Correct instruction
0x1000A3EF	0xA3EF	0xB9	cbnz r2, 0x1000A428	0xB1	cbz r2, 0x1000A428

Patch.

File Offset	VA	Original Bytes	Patched Bytes
0xA3EF	0x1000A3EF	DA B9	DA B1

The anti-debug obstacle. The implant reads CoreDebug DHCSR at `0xE00EDF0` and returns early while a probe is attached, which suppresses the check-in and the task handler:

```

1000a486:      f04f 23e0     mov.w   r3, #3758153728 @ 0xe000e000
1000a48a:      f8d3 3df0     ldr.w   r3, [r3, #3568] @ 0xdf0
1000a48e:      079b          lsls    r3, r3, #30
1000a490:      d114          bne.n   1000a4bc <implant_tick+0x54>

```

```

1000a2b0:      f04f 23e0     mov.w   r3, #3758153728 @ 0xe000e000
1000a2b4:      f8d3 3df0     ldr.w   r3, [r3, #3568] @ 0xdf0
1000a2b8:      079b          lsls    r3, r3, #30
1000a2ba:      d109          bne.n   1000a2d0 <implant_handle_command+0x2c>

```

The shift `lsls r3, r3, #30` keeps bit 1 (`C_HALT`) and bit 0 (`C_DEBUGEN`) in the carry and sign positions, and the `bne` returns early when either bit is set. The guard is identical in both images, so it is an analysis obstacle, not one of the four graded defects.

Defeating the anti-debug. Clear the debug bits in the register as seen by the target, or patch the read in a scratch copy. The register is only a view of debug state, so clearing it makes the attach test see no probe. Show the command sequence, not a fabricated transcript; record what the target actually does:

```
arm-none-eabi-gdb ACT-VII.elf
(gdb) target extended-remote /dev/cu.usbmodemXXXX
(gdb) monitor reset halt
(gdb) break implant_init
(gdb) continue
(gdb) set {unsigned int}0xE000EDF0 = 0
(gdb) break *0x1000A424
(gdb) continue
(gdb) x/4xb 0x103FF000
```

To observe the boot write on the compromised image, break after the flash program at `0x1000A424` (`msr PRIMASK, r4`) in `implant_init`, then read the reserved sector at `0x103FF000` and confirm the first byte is `C7`. To observe the check-in and the task handler, clear the debug bits (or patch the `ldr.w` at `0x1000A48A` in a scratch copy to load a zero constant) and let `implant_tick` run. The scratch copy is for observation only; the shipped artifact is patched at the defect.

Why no marker is written. Under the compromised `cbnz`, the marker gate is inverted: the write path is taken when the gate is clear, so the first boot writes `0xC7` to `0x103FF000`. After the patch, `cbz` returns while the gate is clear, so the flash erase and program at `0x1000A414` and `0x1000A420` are never reached and the sector stays blank. The marker is the durable state that re-arms the payload handler and the check-in on every later boot, and the reserved sector sits outside the program region a firmware reflash writes, which is why the marker survives a reflash and why the gate must be fixed in code, not only erased on the bench.

Grading Rubric (1-to-1 Mapping)

Criterion	Points	Full Credit (Answer Key)
[DOCUMENT] Located the bot marker branch at <code>0x1000A3EF</code>	5	Address and inlined <code>implant_init</code> path identified
[DOCUMENT] Documented the CoreDebug DHCSR anti-debug and how it is defeated under GDB	5	<code>0xE000EDF0</code> , <code>C_DEBUGEN</code> and <code>C_HALT</code> , and a real defeat method
[DOCUMENT & PATCH] Patched <code>0xB9</code> to <code>0xB1</code> so no marker is written to <code>0x103FF000</code>	7	Byte <code>0xB9</code> changed to <code>0xB1</code>
[DOCUMENT] Explained the reserved sector <code>0x103FF000</code> and the bot marker byte <code>0xC7</code>	3	Marker, reserved sector, write-once first run

Instructor Notes & Assembly

- The infect path is inlined into `implant_init`; there is no standalone `implant_infect` symbol in the stripped image.
- The condition byte is the high byte at `0xA3EF`; the correct halfword is `b1da` for `cbz` and the compromised halfword is `b9da`, so the on-disk bytes are `DA B1` for the fix and `DA B9` for the compromise.
- The marker byte is `ANDON_IMPLANT_MARKER_BYTE` (`0xC7`), the reserved sector is `ANDON_IMPLANT_RESERVE_ADDR` (`0x103FF000`), and the marker gate is at `0x20013CFC`.

- The DHCSR address is `ANDON_IMPLANT_DHCSR_ADDR ( 0xE000EDF0 )`; bit 0 is `ANDON_IMPLANT_DHCSR_DEBUGEN ( 0x00000001 )` and bit 1 is `ANDON_IMPLANT_DHCSR_HALT ( 0x00000002 )`. The anti-debug is identical in both images, so it is an analysis obstacle, not one of the four graded defects.
- Grade the GDB point on a real command sequence and the correct observed code path, not on a memorized register dump. Accept either clearing the bits with GDB or patching the read in a scratch copy.
- A common failure is patching the shipped artifact at `0xA3EF` before observing the marker. The order matters: defeat the anti-debug, observe, then patch.

Task 5: Bug #4 The Fault-Clear Authorization (20 points)

Solution

Locate the branch. In `control_handle_frame` (starts at `0x100074D4`) the authorization branch is at file offset `0x7541` (VA `0x10007541`). The corrected image is:

```

10007536:    990a      ldr    r1, [sp, #40]    @ 0x28
10007538:    4808      ldr    r0, [pc, #32]    @ (1000755c <control_handle_frame+0x88>)
1000753a:    aa06      add    r2, sp, #24
1000753c:    f000 f8a2  bl     10007684 <andon_auth_apply>
10007540:    b128      cbz    r0, 1000754e <control_handle_frame+0x7a>
10007542:    4a07      ldr    r2, [pc, #28]    @ (10007560 <control_handle_frame+0x8c>)
10007544:    4b07      ldr    r3, [pc, #28]    @ (10007564 <control_handle_frame+0x90>)
10007546:    7014      strb   r4, [r2, #0]
10007548:    801d      strh   r5, [r3, #0]
1000754a:    b017      add    sp, #92 @ 0x5c
1000754c:    bd30      pop    {r4, r5, pc}
1000754e:    2000      movs   r0, #0
10007550:    b017      add    sp, #92 @ 0x5c
10007552:    bd30      pop    {r4, r5, pc}

```

Instruction decode. After the sealed frame is opened, the command byte is range-checked by the `cmp / bhi` pair at `0x10007514 / 0x10007518` and the zone is range-checked against the 0 to 16 band by the `cmp / bhi` pair at `0x1000751A / 0x1000751C`. `andon_auth_apply` verifies the anti-replay sequence window and the authenticated-state tag and returns its authorization verdict in `r0`. The branch at `0x10007540` decides whether the command may reach the applied command and zone. The correct code rejects a failed or replayed authorization, so the branch at `0x10007540` must be `cbz ( 0xB1 )` to the `0x1000754E` reject path, which returns zero. Only a true verdict falls through to `strb r4, [r2, #0]` and `strh r5, [r3, #0]`, which write the accepted command at `0x20013CF2` and the zone at `0x20013CE2`. The condition byte is the high byte at `0x10007541`.

Address	File offset	Compromised byte	Compromised instruction	Correct byte	Correct instruction
<code>0x10007541</code>	<code>0x7541</code>	<code>0xB9</code>	<code>cbnz r0, 0x1000754E</code>	<code>0xB1</code>	<code>cbz r0, 0x1000754E</code>

Patch.

File Offset	VA	Original Bytes	Patched Bytes
<code>0x7541</code>	<code>0x10007541</code>	28 B9	28 B1

Why the command now requires authorization. Under the compromised `cbnz`, the verdict is inverted: a failed or replayed authorization falls through to the stores at `0x10007546`, while a genuine authorization branches to the reject path and returns zero. After the patch, `cbz` sends a false verdict to the reject path at `0x1000754E`, so an unauthenticated command, a forged command, and a replayed captured command all fail

before the command byte and zone are applied. A legitimate authorized command still returns true and applies. The rest of the path is correct: the envelope is opened under the field key, the command byte is checked against `ANDON_COMMAND_FAULT ( 0x01 )`, `ANDON_COMMAND_CLEAR ( 0x02 )`, and `ANDON_COMMAND_ACK ( 0x03 )`, and the zone is checked against the band `0` to `16`.

Grading Rubric (1-to-1 Mapping)

Criterion	Points	Full Credit (Answer Key)
[DOCUMENT] Located the fault-clear authorization branch at <code>0x10007541</code>	5	Address and function (<code>control_handle_frame</code>) identified
[DOCUMENT] Documented the authorization verdict inversion and the branch condition	5	Reject when the verdict is false
[DOCUMENT & PATCH] Patched <code>0xB9</code> to <code>0xB1</code> so failed and replayed authorizations are rejected	7	Byte <code>0xB9</code> changed to <code>0xB1</code>
[DOCUMENT] Explained why an unauthenticated or replayed fault or clear envelope must be rejected	3	The applied command must see only an authorized verdict

Instructor Notes & Assembly

- The condition byte is the high byte at `0x7541`; the correct halfword is `b128` for `cbz` and the compromised halfword is `b928`, so the on-disk bytes are `28 B1` for the fix and `28 B9` for the compromise.
- `andon_auth_apply` performs the monotonic anti-replay check and the authenticated-state tag, so this branch is the verdict for both freshness and state integrity.
- Full credit requires the inversion explanation: the compromised build accepts a false verdict and rejects a true one.
- Point out that the rest of the fault command path is correct. Only the verdict seam was broken.
- This is the defect that is a policy defect rather than an implant behavior, and it is the one a defender would fix first in production.

Task 6: Export and Verify (10 points)

Solution

Export. In Ghidra, `File -> Export Program...`, choose `Binary Format`, and save as `ACT-VII_fixed.bin`. The shipped image is 51,196 bytes.

Convert.

```
python uf2conv.py ACT-VII_fixed.bin --base 0x10000000 --family 0xe48bff59 --output ACT-VII_fixed.uf2
```

If `uf2conv.py` is not in the working directory, use the copy shipped with the project repository. The UF2 for ACT-VII is 102,912 bytes.

Verify.

```
python scripts/verify_ctf.py
```

Expected result:

10/10 checks passed

Hardware proof. Flash `ACT-VII_fixed.uf2` in BOOTSEL mode and confirm:

- the reserved sector at `0x103FF000` stays blank after a boot;
- the node no longer emits a `C2V1` check-in frame on the four-tick interval;
- a `C2V1` task frame no longer arms the payload or runs a task;
- an unauthenticated command and a replayed captured command are rejected before the command and zone are applied;
- a legitimate authorized command still applies, and the local clear request, the fault-clear button, and the fail-safe divert policy still behave.

Summary of all patches.

#	Bug	File Offset	Flash Address	Original Byte	Patched Byte
1	The C2 Check-In	<code>0xA47F</code>	<code>0x1000A47F</code>	B9	B1
2	The Task Handler	<code>0xA2A9</code>	<code>0x1000A2A9</code>	B9	B1
3	The Bot Marker	<code>0xA3EF</code>	<code>0x1000A3EF</code>	B9	B1
4	The Fault-Clear Authorization	<code>0x7541</code>	<code>0x10007541</code>	B9	B1

Reflection mapping. The four defects map to real control-system failures:

Defect	Real-world failure
The C2 Check-In	A payload registers with a listener the design never named, so a device that looks healthy is already on someone else's network.
The Task Handler	A node executes data it did not authorize, so a fleet becomes obedient to whoever holds the listener.
The Bot Marker	A payload writes a durable marker to a reserved sector, so the state that re-arms it survives remediation.
The Fault-Clear Authorization	An inverted verdict lets an unauthenticated or replayed command change a physical divert decision and zone.

Grading Rubric (1-to-1 Mapping)

Criterion	Points	Full Credit (Answer Key)
[PATCH] Exported <code>ACT-VII_fixed.bin</code> from Ghidra	2	Valid patched binary
[PATCH] Converted to <code>ACT-VII_fixed.uf2</code> with the correct base and family	2	<code>--base 0x10000000 --family 0xe48bff59</code>
[DOCUMENT] <code>scripts/verify_ctf.py</code> passes and hardware proves the correct behavior	3	Verifier passes and the hardware proof is shown

Criterion	Points	Full Credit (Answer Key)
[DOCUMENT] Reflection maps each of the four defects to a real-world control-system failure	3	Specific mapping for all four

Instructor Notes & Assembly

- Confirm the exported image differs from `ACT-VII.bin` in exactly the four bytes in the table; `scripts/verify_ctf.py` checks this and the SHA-256 values.
- Confirm the UF2 conversion used base `0x10000000` and family `0xe48bff59`.
- The shipped image is 51,196 bytes; the corrected image must be the same size because every patch is in place.
- Grade the reflection on specificity, not length: each of the four defects should name a concrete control-system consequence.
- Remind students that the anti-debug is not patched out of the shipped artifact; only the four defect bytes change.

How To Breadboard

Device	Pin on device	Pico 2 GPIO	Notes
DHT11 line temperature sensor	DATA	GP4	10 kOhm pull-up to 3.3 V if the module needs it
1602 LCD	SDA	GP2	I2C1, backpack address <code>0x27</code>
1602 LCD	SCL	GP3	I2C1, 100 kHz
1602 LCD	VCC / GND	VBUS 5 V / GND	The backpack needs 5 V, not 3.3 V
RYLR998	RX	GP8 (Pico TX)	UART1, 115200, network ID 18
RYLR998	TX	GP9 (Pico RX)	UART1
IR receiver	OUT	GP5	VS1838B, internal pull-up enabled
Line diverter servo	signal	GP14	PWM 50 Hz; 1000 uF bulk cap across servo 5 V and GND
Red LED	anode	GP16	FAULT, 220 to 330 ohm to GND
Yellow LED	anode	GP17	ACK PENDING, 220 to 330 ohm to GND
Green LED	anode	GP18	LINE OK, 220 to 330 ohm to GND
Fault-clear button	leg 1	GP15	Internal pull-up; leg 2 to GND, never to 3.3 V
Onboard LED	built in	GP25	Heartbeat
Debug Probe	SWCLK / SWDIO / GND	debug header	For GDB only

Use 3.3 V logic on every GPIO. The only 5 V connection is the LCD backpack supply and the servo rail. Keep the 1000 uF capacitor on the servo rail to absorb the SG90 current spike.

Complete Grading Summary

Task	Title	Points
Task 1	Setup and Initial Analysis	10
Task 2	Bug #1 The C2 Check-In	20
Task 3	Bug #2 The Task Handler	20
Task 4	Bug #3 The Bot Marker	20
Task 5	Bug #4 The Fault-Clear Authorization	20
Task 6	Export and Verify	10
TOTAL		100

Instructor Notes

Safety: Use only the supplied Pico 2, Debug Probe, and firmware. Never connect the exercise to an operational factory network, a manufacturing execution system, a building-management system, a public network, a military system, or a third-party device.

Common Student Mistakes

- Patching the low byte of the branch at `0xA47E`, `0xA2A8`, `0xA3EE`, or `0x7540` instead of the condition byte at `0xA47F`, `0xA2A9`, `0xA3EF`, or `0x7541`.
- Reading the check-in gate or the task gate backwards and believing the corrected build still registers or still executes a task.
- Searching for a standalone `implant_infect`, `implant_check_in`, or `implant_tasking_frame` symbol and missing that all three are inlined into `implant_init`, `implant_tick`, and `implant_handle_command`.
- Treating the CoreDebug `DHCSR` anti-debug as a defect and trying to patch it, when it is identical in both images and is an analysis obstacle.
- Patching the shipped artifact before observing the marker write, so the bot is never demonstrated.
- Reversing the authorization explanation: under the compromise the accept path is taken when the verdict is false.
- Confusing `cbz` and `cbnz` on the two clearing gates.
- Forgetting that the fix for the marker is two parts: the patch and the reserved-sector erasure.
- Forgetting the UF2 conversion or using the wrong family flag.
- Fabricating a GDB session instead of showing the command sequence and the real observed code path.

Partial Credit Guidelines

- Award partial credit for a correct address without the correct byte, or a correct byte without the address.
- Award partial credit for documented before/after bytes without the control-flow explanation, or vice versa.

- Award partial credit for a correct GDB command sequence without a clear statement of the observed code path, or the observation without the commands.
- Award partial credit for a correct anti-debug explanation without a working defeat method, or a working method without the explanation.
- Award partial credit for naming the reserved sector and the marker without the persistence lesson, or the lesson without the addresses.
- Award no credit for patches that alter any byte outside the four documented offsets, and no credit for a fabricated GDB session.

Appendix: Expected Binary Diff

These offsets are from the compiled image loaded at `0x10000000`.

```
--- ACT-VII.bin (compromised)
+++ ACT-VII_fixed.bin (corrected)

Offset 0x00007541: B9 -> B1 (cbnz r0, 0x1000754E -> cbz r0, 0x1000754E)
Offset 0x0000A2A9: B9 -> B1 (cbnz r3, 0x1000A2D0 -> cbz r3, 0x1000A2D0)
Offset 0x0000A3EF: B9 -> B1 (cbnz r2, 0x1000A428 -> cbz r2, 0x1000A428)
Offset 0x0000A47F: B9 -> B1 (cbnz r3, 0x1000A4BC -> cbz r3, 0x1000A4BC)
```

#	Bug	File Offset	Flash Address	Original Bytes	Patched Bytes
1	The C2 Check-In	0xA47F	0x1000A47F	EB B9	EB B1
2	The Task Handler	0xA2A9	0x1000A2A9	93 B9	93 B1
3	The Bot Marker	0xA3EF	0x1000A3EF	DA B9	DA B1
4	The Fault-Clear Authorization	0x7541	0x10007541	28 B9	28 B1

Four defects, four changed bytes in four instructions: the C2 check-in gate, the task handler gate, the bot marker gate, and the authorization verdict. No other byte in either image differs.